

50. ArrayBuffer aur Binary Arrays

Web dev me binary data zyada tar **files** (banana, upload, download) aur **image processing** me milta hai. JS me classes kaafi hain (`ArrayBuffer`, `Uint8Array`, `DataView`, `Blob`, `File`), isliye thoda confusion hota hai, par samajh aane par simple hai.

`ArrayBuffer` (basic binary object)

Fixed length ka contiguous memory area ka reference.

```
js
let buffer = new ArrayBuffer(16); // 16 bytes, zero se bhare
alert(buffer.byteLength);         // 16
```

Ye `Array` se **bilkul alag** hai:

- Fixed length (badha/ghata nahi sakte)
- Utني hi memory leta hai
- `buffer[index]` se access nahi hota, **view object** chahiye

`ArrayBuffer` ko nahi pata ki andar kya hai, ye sirf raw bytes ka sequence hai.

View objects ("chashme")

View khud kuch store nahi karta, wo sirf bytes ki **interpretation** hai:

- `Uint8Array`: har byte alag number (0 se 255)
- `Uint16Array`: har 2 bytes ek integer (0 se 65535)
- `Uint32Array`: har 4 bytes ek integer (0 se 4294967295)
- `Float64Array`: har 8 bytes ek floating point number

To 16 bytes ko 16 chhote numbers, ya 8 (2-byte), ya 4 (4-byte), ya 2 (8-byte float) ki tarah dekh sakte ho.

```
js
let buffer = new ArrayBuffer(16);
let view = new Uint32Array(buffer);

view.length; // 4
view.byteLength; // 16
view[0] = 123456;
```

TypedArray

`Uint8Array`, `Uint32Array` jaise sab views ka common naam **TypedArray** hai (`new TypedArray` koi actual constructor nahi, sirf umbrella term). Ye regular arrays ki tarah indexes wale aur iterable hain.

Constructor ke 5 variants

```
js
new TypedArray(buffer, [byteOffset], [length]); // buffer par view
new TypedArray(object);                        // array/array-like se copy
new TypedArray(typedArray);                    // dusre typed array se copy
new TypedArray(length);                        // itne elements ka array
new TypedArray();                              // zero length
```

Example:

```
js
let arr = new Uint8Array([0, 1, 2, 3]);          // 4 bytes
let arr16 = new Uint16Array([1, 1000]);
let arr8 = new Uint8Array(arr16);               // arr8[1] = 232 (1000 8 bit
let a = new Uint16Array(4);                     // byteLength = 8
```

Baaki sab (buffer wale ke alawa) me `ArrayBuffer` khud ban jaata hai. Underlying buffer tak: `arr.buffer`, `arr.byteLength`. Isse ek view se dusra view bana sakte ho:

```
js
let arr8 = new Uint8Array([0, 1, 2, 3]);
let arr16 = new Uint16Array(arr8.buffer); // same data par dusra view
```

Typed arrays ki list

- `Uint8Array`, `Uint16Array`, `Uint32Array`: unsigned integers (8/16/32 bit)
 - `Uint8ClampedArray`: 8-bit, assign karne par "clamp" karta hai
- `Int8Array`, `Int16Array`, `Int32Array`: signed (negative bhi)
- `Float32Array`, `Float64Array`: signed floating-point (32/64 bit)

`int8` jaisa single-value type JS me nahi hota. `Int8Array` bhi bas `ArrayBuffer` par ek view hai.

Out-of-bounds behavior

Range se bahar value likhne par **error nahi**, extra bits **kat jaate hain** (number modulo 2^8 jaisa):

js

```
let uint8array = new Uint8Array(16);
uint8array[0] = 256; // 0 (256 = 1000000000, sirf daayi 8 bits)
uint8array[1] = 257; // 1
```

`Uint8ClampedArray` alag: 255 se badi value par `255`, negative par `0` (image processing me useful).

TypedArray methods

Regular array jaise: `iterate`, `map`, `slice`, `find`, `reduce`, etc.

Nahi kar sakte:

- `splice` nahi (delete nahi kar sakte, sirf zero assign)
- `concat` nahi

2 extra methods:

- `arr.set(fromArr, [offset])`: `fromArr` ke elements `offset` se copy
- `arr.subarray([begin, end])`: **bina copy kiye** same data par naya view (`slice` copy karta hai)

`DataView`

Ek super-flexible **untyped view**: kisi bhi offset par kisi bhi format me data access.

- Typed arrays me format constructor me fix hota hai. `DataView` me **method call ke waqt** format chunte hain: `.getUint8(i)`, `.getUint16(i)`, `.getUint32(i)`.

js

```
new DataView(buffer, [byteOffset], [byteLength])
```

Ye khud buffer nahi banata, buffer pehle se chahiye.

js

```
let buffer = new Uint8Array([255, 255, 255, 255]).buffer;
let dataView = new DataView(buffer);

dataView.getUint8(0);    // 255
dataView.getUint16(0);   // 65535
dataView.getUint32(0);   // 4294967295
dataView.setUint32(0, 0); // sab bytes 0
```

Ek hi buffer me **mixed formats** (jaise 16-bit integer + 32-bit float ki pairs) ho to bahut kaam aata hai.

Summary

- `ArrayBuffer` = core object (fixed memory). Kaam karne ke liye **view** chahiye.
- View: `TypedArray` (`Uint8Array`, `Int16Array`, `Float32Array`...) ya `DataView`.
- Zyaadatar cases me typed array seedha banate hain aur `ArrayBuffer` andar chhupa rehta hai (`.buffer` se mil jaata hai).
- Do umbrella terms:
 - `ArrayBufferView`: sab views ke liye
 - `BufferSource`: `ArrayBuffer` ya `ArrayBufferView` (yani "koi bhi binary data")

Task: Typed arrays jodna (`concat`)

```
js

function concat(arrays) {
  let totalLength = arrays.reduce((acc, value) => acc + value.length, 0);
  let result = new Uint8Array(totalLength);
  if (!arrays.length) return result;

  let length = 0;
  for (let array of arrays) {
    result.set(array, length); // agla array pichhle ke turant baad
    length += array.length;
  }
  return result;
}
```